Architectural Blueprint for Mapping the Bruhat Bengaluru Mahanagara Palike (BBMP) Act to the Akoma Ntoso LegalDocML Standard

The Transition to Machine-Readable Governance and the Nyayabandhu Mandate

The modernization of legislative infrastructure requires a fundamental shift from static, unstructured document repositories to dynamic, machine-readable ecosystems. Historically, legislative texts in India, including massive municipal frameworks like the Bruhat Bengaluru Mahanagara Palike (BBMP) Act, 2020, have been published in "instructive" or "passive" formats[1]. These documents typically manifest as scattered, fractured PDF files housed across various e-Gazette portals without underlying standardization[1]. This fragmented architecture creates profound systemic inefficiencies, forcing citizens, legal professionals, corporate compliance teams, and the judiciary to manually cross-reference and "stitch" together principal acts with decades of subsequent amendments[1]. The financial and temporal costs of this passive status quo are immense; individual High Courts expend millions annually on private, proprietary legal databases just to access updated, consolidated laws, while subordinate judiciaries often lack access to critical, up-to-date subordinate legislation entirely[1].

The Nyayabandhu initiative aims to dismantle this paradigm by engineering a Digital Public Infrastructure (DPI) dedicated to legislative information[1]. The initiative centralizes the standardization of legislative texts using the Akoma Ntoso XML standard, transforming static images and unstructured text into "Living Data"[1]. The immediate technical imperative within Phase 1 of this project involves establishing an automated data pipeline capable of parsing complex Indian legislation, specifically targeting the English version of the BBMP Act, 2020, before subsequently modeling its regional Kannada translation[1].

The BBMP Act serves as an optimal candidate for this architectural transition due to its vast scope, deep hierarchical complexity, and critical governance role. Enacted to replace the structurally inadequate Karnataka Municipal Corporations Act, 1976, the BBMP Act provides the legal scaffolding for the governance of Bengaluru, detailing everything from mayoral authorities and zonal committee formations to infrastructure cesses and grievance redressal mechanisms[2]. Mapping this comprehensive piece of legislation into the Akoma Ntoso standard requires a highly nuanced understanding of both the LegalDocML schema and the specific idiosyncrasies of Indian legislative drafting. This report provides an exhaustive, systemic blueprint for executing this transformation, addressing required schema customizations, the

implementation of artificial intelligence-driven extraction pipelines, and the subsequent technological pathways necessary for multilingual parity.

Theoretical Foundations of the Akoma Ntoso Architecture

Akoma Ntoso, officially ratified as the OASIS LegalDocumentML (LegalDocML) standard, is an internationally recognized XML vocabulary specifically engineered to represent parliamentary, legislative, and judicial documents[3]. The standard provides a sophisticated, extensible schema that separates a document's semantic meaning, hierarchical structure, and metadata from its visual presentation[4]. By embedding legal knowledge into machine-readable syntax, Akoma Ntoso enables automated point-in-time version control, sophisticated cross-referencing, and seamless interoperability across diverse governmental bodies and legal-tech platforms[1].

At the philosophical and structural core of Akoma Ntoso is the Functional Requirements for Bibliographic Records (FRBR) conceptual ontology[7]. The FRBR model organizes legal documents into distinct layers of abstraction, which is essential for managing the temporal evolution and multilingual realities of Indian legislation[5]. The architecture classifies documents according to four primary entities:

FRBR Entity	Architectural Definition	Application to the BBMP Act
Work	The abstract intellectual creation or legal concept, independent of language, format, or physical manifestation[7].	The underlying legal concept of the "Bruhat Bengaluru Mahanagara Palike Act, 2020," regardless of whether it is read in English or Kannada[7].
Expression	A specific intellectual or artistic realization of the Work. This includes translations and specific chronological versions (amendments)[7].	The specific English text of the Act upon its initial passing represents one Expression. The Kannada translation represents a parallel Expression of the same Work[5].
Manifestation	The physical or digital carrier that embodies an Expression. This dictates the file format[7].	The physical printed Gazette, the uploaded PDF, or the serialized Akoma Ntoso XML file are all

		distinct Manifestations[7].
Item	A single exemplar or physical instance of a Manifestation, typically pointing to a specific storage location or URL[8].	The exact digital file hosted at a specific server endpoint, such as the XML file accessed via the Nyayabandhu API[9].

This multidimensional architecture allows for independent versioning and localization[7]. Under the FRBR model, when the BBMP Act is mapped into Kannada, it does not require a completely new structural framework; rather, it is registered as a new Expression tied to the original Work, ensuring absolute structural and semantic parity across languages[5].

Structural Anatomy of the BBMP Act, 2020

Translating the BBMP Act into XML requires a preliminary dissection of its structural taxonomy. Commonwealth and Indian legislative drafting traditions utilize deeply nested, highly formalized hierarchies to organize legal provisions[2]. The parser mapping the BBMP Act must recognize and accurately compartmentalize these structural layers.

The BBMP Act comprises hundreds of pages of dense legal instructions, structured categorically from broad administrative chapters down to granular clauses and schedules[2]. The highest organizational level consists of twenty-five distinct Chapters, denoted by Roman numerals. For example, Chapter I handles Preliminary provisions and definitions, Chapter II outlines Corporation Authorities (such as the Mayor, Chief Commissioner, and Ward Committees), and Chapter XIII governs Taxes[2]. Beneath the Chapter level, the document is divided into a continuous numerical sequence of Sections, extending from Section 1 to Section 376[2]. Unlike some international jurisdictions where section numbers restart with each new chapter, the Indian tradition maintains a persistent sequence throughout the entire body of the Act[11].

Sections are further subdivided into parenthetical Sub-sections, such as (1) and (2)[2]. When a sub-section requires additional granularity, it drops into parenthetical alphabetic Clauses, such as (a) and (b), and subsequently into lower-case Roman numeral Sub-clauses, such as (i) and (ii)[2]. Additionally, the Act features highly specialized semantic zones. Section 2 serves as a centralized glossary containing sixty-nine distinct definitions, ranging from the definition of an "Annual Financial Statement" to the boundaries of "Zones of the Corporation"[2]. Finally, appended to the end of the Act are Schedules, which act as critical addendums detailing specific procedural tables, taxation limits, and geographical coordinates[11].

Serializing the English BBMP Act: Mapping Strategy and Syntax

The primary task of mapping the English text into the Akoma Ntoso format involves translating

the aforementioned taxonomy into the strict syntactic rules of the LegalDocML schema. Akoma Ntoso mandates that every document must share the same root element, <akomaNtoso>, beneath which the specific document type is declared[9]. For the BBMP Act, the appropriate document type is <act>, which represents an autonomous, pre-existing document issued by a defined authority[9].

Constructing the Metadata and FRBR Identifiers

Immediately following the <act> declaration, the parser must generate the <meta> block[10]. This section is completely separated from the readable content of the law and is utilized by machine-driven processes to understand the document's origin, classification, and lifecycle[10]. The most critical component of the metadata block is the FRBR Identification matrix, which assigns a permanent, universally recognizable URI to the document[14].

The Akoma Ntoso naming convention dictates that the URI must be algorithmically constructed based on the jurisdiction, document type, date of enactment, and language[14]. The URI construction for the English BBMP Act is formulated as follows:

Conceptual Component	URI String Representation	Rationale based on LegalDocML Standard
Domain and Jurisdiction	/akn/in/karnataka	Identifies the Akoma Ntoso (akn) namespace, the country (in for India), and the specific state jurisdiction (karnataka)[14].
Document Type	/act	Specifies that the document is a primary legislative act[9].
Temporal Identifier	/2020	Denotes the year of enactment[14].
Number or Name	/bbmp_act	Uses a unique string or the official act number to differentiate it from other acts passed in the same jurisdiction and year[14].
Language Expression	/eng@	Specifies the language using ISO 639-2 Alpha-3 formatting (eng), combined

		with the @ symbol to denote the Expression level[14].
Manifestation	/main.xml	Indicates the digital format of the file being accessed[7].

This results in a full FRBR URI: /akn/in/karnataka/act/2020/bbmp_act/eng@/main.xml[14]. In addition to the identification block, the metadata section must include a <publication> element indicating the date the Act was published in the official Gazette, a <lifecycle> block to trace the chronological history of the document as amendments are introduced, and a <classification> block utilizing Top Level Class (TLC) references[13]. TLC references are ontological anchors that classify the document by subject matter, making it highly discoverable for legal researchers[13].

Hierarchical Element Mapping

Following the metadata, the document structure transitions into the <preface> and the <body>[12]. Akoma Ntoso provides a highly descriptive vocabulary for structural containers that map neatly onto the Indian drafting style. The automation script mapping the BBMP Act must enforce strict containment rules, ensuring that text only resides within the lowest logical elements[17].

The hierarchical mapping of the BBMP Act is executed through the following sequence:

BBMP Act Hierarchy	**Akoma Ntoso XML Element**	**Example Identifier (@eId)**
Act Title and Introductory Matter	<preface>	preface
Substantive Text Wrapper	<body>	body
Chapter (e.g., Chapter II)	<chapter>	chp_II
Section (e.g., Section 79)	<section>	sec_79
Sub-section (e.g., (1))	<subsection>	sec_79-subsec_1
Clause (e.g., (a))	<clause>	sec_79-subsec_1-clause_a
Sub-clause (e.g., (i))	<subclause>	sec_79-subsec_1-clause_a-subclause_i

Within these structural wrappers, the actual prose of the legislation is contained within <content> elements, which are further broken down into paragraph <p> tags[12]. The Akoma Ntoso schema strictly prohibits placing readable text directly inside a <chapter> or <section> without first nesting it inside a <content> and <p> block, ensuring that every character of the law is precisely addressable[4].

Internal Node Identifiers: Managing Temporal Dynamics

A cornerstone of machine-readable law is the ability to track the evolution of specific provisions over time[10]. The Nyayabandhu system requires that the BBMP Act mapping utilize two distinct identifier attributes on every structural element: the Expression Identifier (@eId) and the Work Identifier (@wId)[14].

The @eId reflects the absolute structural path of an element within the current version of the document[14]. If the BBMP Act contains a <section eId="sec_5">, this identifier confirms that it is structurally the fifth section of the text. However, legislative amendments frequently insert or repeal sections, altering the structural flow[16]. If a new section is inserted between Section 4 and Section 5, the new section assumes the structural position of Section 5, and the original Section 5 becomes Section 6. The @eId must update to reflect this new reality[16].

Conversely, the @wId serves as the permanent, immutable conceptual identifier of the provision[14]. Regardless of how the document is renumbered or reorganized in future amendments, the @wId remains constant, allowing tracking algorithms to isolate the chronological history of a specific legal mandate[16]. For the initial mapping of the base BBMP Act, the @eId and @wId values generated by the extraction script will be identical. Divergence only occurs when Phase 2 of the Nyayabandhu project introduces the programmatic application of amendment acts[1].

Semantic Enrichment and Ontological Tagging

To elevate the BBMP Act from a merely structured document to a highly semantic knowledge graph, the parsing pipeline must identify and annotate inline legal concepts[4]. This semantic enrichment is critical for the APIs that will serve the data to legal-tech startups and compliance platforms[1].

The BBMP Act contains numerous internal and external cross-references. When Section 79 mandates the allocation of funds in accordance with Section 78, the text "Section 78" must be wrapped in a <ref> tag[2]. The href attribute of this tag points directly to the #sec_78 identifier within the document, creating a cohesive, navigable legal map[1]. When the Act refers to external legislation, such as the Karnataka Municipal Corporations Act, 1976, the <ref> tag must point to the absolute FRBR URI of that external document[2].

Furthermore, Section 2 of the BBMP Act, which serves as the definitional dictionary, requires precise modeling. The parser must wrap the entire section in a <blockList> element, treating each of the sixty-nine definitions as an individual <item>[2]. Within the text of the item, the

specific term being defined (e.g., "Bio-medical waste") is enclosed in a <def> tag[2]. This semantic marker allows external systems to algorithmically build a glossary or instantly display definition tooltips when the term appears later in the document[15]. Additional semantic tags must be applied to temporal constraints, wrapping textual dates (e.g., "five years from the date appointed") in <date> elements featuring normalized ISO-8601 attributes[2].

Customizing the Schema: Required Format Changes for Indian Legislation

The user inquiry explicitly requests an analysis of what changes are required to the Akoma Ntoso format to accommodate the BBMP Act. The LegalDocML standard utilizes a two-tiered architectural philosophy consisting of a highly permissive "General Schema" and highly prescriptive "Detailed Schemas"[21]. Because the General Schema is designed to be universally descriptive and flexible across global legal traditions, no fundamental alterations to the core XSD (XML Schema Definition) are strictly necessary[4].

However, enforcing data quality and structural predictability requires generating an Application Profile—a localized, restricted subschema tailored explicitly to Indian drafting conventions[4]. This is achieved using the Akoma Ntoso SubSchema Generator, a web-based tool and REST API that allows architects to select specific vocabulary modules and implement XSD 1.1 co-constraints[4]. The parsing of the BBMP Act necessitates several critical customizations via this mechanism.

The Schedule Module

A major point of divergence between European and Commonwealth legal drafting is the treatment of end-matter[11]. In the European tradition, an Annex is treated as a separate, attached document. In the Indian tradition, a Schedule is an intrinsic, inseparable component of the principal Act's body[11]. Historically, mapping Indian schedules into Akoma Ntoso proved difficult, forcing developers to rely on generic <hcontainer> tags[11].

However, with the release of Akoma Ntoso 3.0, native <schedule> and <subschedule> elements were introduced[11]. The Application Profile for the BBMP Act must explicitly import the "Collection of smaller documents" module to enable the <schedule> tag[12]. This customization ensures that the intricate lists, infrastructural mandates, and geographical boundaries appended to the BBMP Act are serialized cleanly without triggering validation errors[12].

Customizing Provisos and Explanations

Indian statutes are heavily reliant on the use of "Provisos" (clauses beginning with "Provided that...") and "Explanations" inserted immediately following a substantive sub-section[2]. The Akoma Ntoso vocabulary does not contain dedicated tags specifically named <proviso> or <explanation>.

To map these constructs without altering the base standard, the SubSchema Generator must be configured to utilize generic containers paired with specific naming attributes[17]. The

Application Profile must dictate that provisos are mapped using the generic <block> or <blockList> element with the attribute @name="proviso". This restriction ensures that the AI extraction pipeline maintains absolute consistency, preventing different parsing scripts from arbitrarily assigning different tags to identical legal constructs[17].

Complex Tabular Data Integration

The schedules embedded within Indian legislation frequently rely on highly complex, merged tabular structures to define tax brackets, zoning coordinates, and administrative hierarchies. The default Akoma Ntoso schema supports basic tabular representation through an integrated HTML table module (XHTML 2.0)[12].

However, the specific visual presentation of these tables often conveys implicit legal meaning. The format must be customized to permit external namespace extensions[4]. By expanding the schema to accept external CSS stylesheets or advanced formatting grids, the Nyayabandhu system can preserve the exact visual logic of the BBMP Act's taxation limits while retaining the machine-readability of the underlying data[4].

Engineering the Automation Pipeline: Execution Methodologies

The most formidable technical challenge outlined in the Nyayabandhu Phase 1 concept note is the "Stitching Gap"[1]. Currently, there is no automated, open-source pipeline tailored to the complex semantics of Indian legislation capable of ingesting PDF documents, understanding their legal logic, and emitting standardized XML[1]. Commercial string-comparison software utilized by large law firms fails to adequately address this requirement[1]. Therefore, generating the English serialization of the BBMP Act requires the deployment of a highly sophisticated, multi-tiered algorithmic pipeline.

Legacy Methodologies: Rule-Based Backtracking Parsers

Early initiatives in the legal informatics space, such as the "Laws of India" project managed by the Nyaaya initiative and hosted on Indian Kanoon's Indigo platform, utilized rule-based parsers to generate Akoma Ntoso XML[22]. The Indigo platform, a Django-based Python web application utilizing PostgreSQL, relies heavily on a library known as Slaw[23].

Written in Ruby, Slaw utilizes Treetop grammars to act as a backtracking parser[25]. It ingests plain text, analyzes it against a hardcoded set of grammatical rules tailored to specific legal traditions (such as South African by-laws), and builds a syntax tree that serializes into XML[25]. While highly deterministic and reliable for perfectly clean, structurally consistent text, rule-based parsers are exceedingly brittle[25]. They fail catastrophically when presented with optical character recognition (OCR) errors, oddly wrapped lines, nested list numbering anomalies, or slight deviations in legislative drafting language[1]. Given the highly unstructured and often poorly scanned nature of Indian Gazette PDFs, relying exclusively on legacy parsers like Slaw is insufficient for the Nyayabandhu mandate[1].

Modern AI Methodologies: Large Language Models and Agentic Workflows

To overcome the fragility of rule-based systems and solve the semantic complexities of the Stitching Gap, the extraction pipeline must integrate Natural Language Processing (NLP) and Large Language Models (LLMs)[1]. The goal of Phase 1 is to programmatically extract the instructional logic of the text, correctly identifying the Target Act, Target Location, Action (Insert, Substitute, Omit), Content, and References[1].

Emerging open-source frameworks, such as the *Schematise* project, provide a functional blueprint for this methodology[19]. Schematise is an LLM-enabled XML generator designed explicitly for Indian statutes, built on the LangChain framework[27]. The execution pipeline for the English BBMP Act utilizing this methodology involves several distinct phases:

1. **Denoising and Spatial OCR**: The raw PDF of the BBMP Act is ingested. Advanced, layout-aware OCR engines must extract the text while preserving spatial relationships, ensuring that tabular data and hierarchical indentations are not flattened into continuous strings[1].
2. **Retrieval-Augmented Generation (RAG)**: The cleaned text is processed through an LLM (utilizing open-source models like LLaMA-2-7B for localized, secure inference, or proprietary APIs like OpenAI GPT-3.5 turbo)[27]. The model operates within a RAG framework, referencing a vector database of properly formatted Akoma Ntoso snippets to contextualize its output[27].
3. **Semantic Structuring via Few-Shot Prompting**: The LLM is guided by few-shot prompts containing meticulously curated examples of Indian legal semantics[27]. The model is instructed to identify that "Section 142" triggers a <section> tag, while "Provided that" triggers a <block name="proviso"> tag[2].
4. **Multi-Agent Validation**: Advanced implementations leverage multi-agent architectures to ensure accuracy[29]. A "Structure Generator" agent maps the hierarchy and extracts headwords. A "Structure Verifier" agent cross-references the generated XML against the original PDF to ensure no clauses were omitted or hallucinated by the LLM. Finally, a "Structure Curator" agent cleans the XML, enforcing the strict validation rules of the OASIS standard[29].

This AI-driven pipeline outputs machine-readable XML, which is then presented through a "Human-in-the-Loop" dashboard. Here, legal domain experts and Section Officers can manually review the AI's output, attesting to its accuracy before the document is committed to the live Nyayabandhu database[1].

The Multilingual Architecture: Modeling the Kannada Translation

Upon the successful serialization of the English BBMP Act, the subsequent technical requirement dictates the mapping of its regional Kannada translation. In the context of Indian

governance, absolute bilingual parity is legally mandated. The architecture must ensure that the Kannada text is not treated as a disconnected, secondary document, but rather as an intrinsically linked equivalent[5].

FRBR Alignment and Expression Serialization

The Akoma Ntoso schema natively resolves multilingual complexities without requiring any alterations to the XML structure itself[5]. Returning to the FRBR conceptual model, the Kannada translation is serialized as a distinct Expression of the exact same abstract Work[7].

The structural taxonomy generated during the English parsing phase acts as the foundational template. A <section eId="sec_5"> mapped in the English XML must map perfectly to a <section eId="sec_5"> in the Kannada XML[5]. This absolute structural alignment ensures that frontend dashboards can instantly toggle between language versions at the paragraph level, providing seamless bilingual navigation for end-users[5].

The metadata block of the Kannada document must reflect its unique Expression status[16]. The <language> tag is set using the ISO 639-2 Alpha-3 code for Kannada (kan)[13]. Correspondingly, the FRBR URI is updated to reflect this expression: /akn/in/karnataka/act/2020/bbmp_act/kan@/main.xml[14]. If architectural preferences require a side-by-side bilingual manifestation within a single XML file, the root document language is set to mul (multiple languages), and the @xml:lang attribute is injected directly into individual structural elements (e.g., <p xml:lang="en"> and <p xml:lang="kn">)[16].

Indic Script OCR and the Bhashini Ecosystem

The primary engineering obstacle in mapping the Kannada BBMP Act is the extraction of the raw text. State-level official gazettes are notoriously difficult to process; they are frequently published as low-resolution scanned images or utilize non-Unicode, legacy proprietary fonts that completely confound standard Latin-based OCR engines like Tesseract[1].

To reliably digitize the Kannada text, the pipeline must integrate specialized Indic OCR models. The ecosystem developed under the Government of India's Bhashini mission and the AI4Bharat initiative provides the necessary infrastructure[31]. AI4Bharat has curated the Bharat Scene Text Dataset (BSTD), a massive benchmark explicitly designed to advance script recognition for Indian languages[30]. By utilizing models fine-tuned on this dataset (such as the PARSeq model) via the Bhashini API or the IndicPhotoOCR SDK, the pipeline can execute highly accurate detection, script identification, and end-to-end recognition on the Kannada gazette PDFs, effectively bypassing the limitations of commercial OCR solutions[30].

Neural Machine Translation for Dynamic Alignment

In scenarios where a digitized Kannada version is unavailable, or where rapid amendments are passed in English and require immediate localization, the pipeline can utilize state-of-the-art Neural Machine Translation (NMT)[34]. Legacy translation models struggle with legal texts because they process information in isolated sentences, stripping away the complex contextual flow required by statutory drafting[34].

To address this, the pipeline can deploy the Sarvam-Translate model, developed in collaboration with AI4Bharat[34]. Built on the Gemma3-4B-IT architecture, Sarvam-Translate is engineered specifically for comprehensive, document-level translation across 22 official Indian languages[34]. Possessing an 8,000-token context window, the model can digest large contiguous blocks of the English BBMP XML, interpret the highly formalized semantic tone, and generate accurate Kannada translations while preserving the surrounding Akoma Ntoso markup tags[34]. Furthermore, all models and datasets utilized in this process are standardized and benchmarked through the Universal Language Contribution API (ULCA), an open, scalable data platform integral to the Bhashini mission, ensuring continuous improvement in translation accuracy[33].

Conclusion

The transformation of the Bruhat Bengaluru Mahanagara Palike (BBMP) Act, 2020 into the Akoma Ntoso LegalDocML standard is a highly technical but imminently achievable imperative for the Nyayabandhu initiative[1]. The Akoma Ntoso framework possesses the inherent structural flexibility required to encapsulate the massive hierarchical depth, complex scheduling, and definitional cross-referencing mechanics embedded within the Act[4]. No fundamental alterations to the global OASIS standard are required; rather, the execution demands the meticulous configuration of an Application Profile using the SubSchema Generator to standardize Indian legislative anomalies, such as provisos and appended schedules[4]. Furthermore, the dual-language requirement of mapping both the English and Kannada texts is elegantly resolved through the standard's underlying FRBR conceptual ontology, which treats language versions as perfectly aligned parallel Expressions of a singular legal Work[7]. While legacy, rule-based parsers are inadequate for processing the unstructured nature of Indian Gazette PDFs, the integration of modern artificial intelligence pipelines bridges the technological gap[1]. By combining layout-aware Indic OCR models, RAG-enabled Large Language Models, and advanced context-aware translation APIs supported by the AI4Bharat and Bhashini ecosystems, the automation pipeline can successfully extract, structure, and serialize the legislation[27]. The successful deployment of this architectural blueprint will not only transform the BBMP Act into a universally interoperable, machine-readable digital asset but will also establish a replicable, open-source technological foundation capable of revolutionizing legislative discoverability across the entire Indian subcontinent[1].

Works cited

1. Updated Project Concept Note - Nyayabandhu (IIIT-H Phase 1).pdf
2. BBMP Act.pdf
3. Towards a country-independent data format: the Akoma Ntoso experience, https://www.semanticscholar.org/paper/Towards-a-country-independent-data-format%3A-the-Vitali-Zeni/8d28ba31a6703f3d6047275d891486e7799f9304
4. Akoma Ntoso: - IRIS, https://cris.unibo.it/retrieve/e1dcb333-5f55-7715-e053-1705fe0a6cc9/balisageVol

24-2019.pdf
5. Akoma Ntoso - Wikipedia, https://en.wikipedia.org/wiki/Akoma_Ntoso
6. Akoma Ntoso: an open standard for harmonising legal resouces - Agora, https://www.agora-parl.org/sites/default/files/agora-documents/University%20of%20Bologna%20-%20Akoma%20Ntoso.%20An%20open%20standard%20-%2011.2008%20-%20EN%20-%20PI.pdf
7. A Temporal FRBR/FRBRoo-Based Model for Component-Level Versioning of Legal Norms, https://arxiv.org/html/2506.07853v1
8. Works, Expressions, Manifestations, Items: An Ontology - The Code4Lib Journal, https://journal.code4lib.org/articles/16491
9. an overview - 2 Akoma Ntoso, https://unsceb-hlcm.github.io/part1/index-13.html
10. Akoma Ntoso | Legis Info, https://legisinfo.com/category/akoma-ntoso/
11. Akoma Ntoso | Legis Info, https://legisinfo.com/tag/akoma-ntoso/
12. AKN Subschema Frontend, https://akn.web.cs.unibo.it/akgenerator/
13. Automatic Transformation of Plain-text Legislation into Machine-readable Format - Eventiotic, https://eventiotic.com/eventiotic/files/Papers/URL/77997450-92c5-4fef-ad5d-254764b9165a.pdf
14. 6 AKN4UN Naming convention - Akoma Ntoso, https://unsceb-hlcm.github.io/part1/index-145.html
15. Indigo principles — Indigo Platform 18.0.0 documentation - Read the Docs, https://indigo.readthedocs.io/en/latest/guide/principles.html
16. 5 AKN4UN Metadata - Akoma Ntoso, https://unsceb-hlcm.github.io/part1/index-110.html
17. 4 AKN4UN: Modelling Content - Akoma Ntoso, https://unsceb-hlcm.github.io/part1/index-54.html
18. The story behind Schematise and a use-case demonstration | Sankalp Srivastava, https://sankalpsrv.in/posts/2024/2024-04-16-Complianalyse/
19. [Demo] Indian Statutes and Laws in Akoma Ntoso Format | Schematise | The Fifth Elephant, https://www.youtube.com/watch?v=GgcHsDJEvs4
20. uslm/USLM-User-Guide.md at main · usgpo/uslm - GitHub, https://github.com/usgpo/uslm/blob/main/USLM-User-Guide.md
21. Introduction to Akoma Ntoso - un desa dpidg, https://publicadministration.un.org/Parliaments/Portals/0/conferences/capetown2006/04%20Fabio%20Vitali%20-%20Introduction%20to%20Akoma%20Ntoso.pdf
22. Module 1 - Session 2 - Data Collection - CivicDataLab, https://civicdatalab.in/Working-with-Data-Workshops/modules/module_1_data_collection/session-2/session-2.html
23. Launching Laws of India in the Akoma Ntoso format - Indian Kanoon Blog, https://blog.indiankanoon.org/2019/11/launching-laws-of-india-in-akoma-ntoso.html
24. GitHub - nyaayaIN/laws-of-india: Laws of India in Akoma Ntoso XML format, https://github.com/nyaayaIN/laws-of-india
25. laws-africa/slaw: Slaw is a lightweight library for rendering and generating Akoma Ntoso acts from plain text and PDF documents. - GitHub,

https://github.com/laws-africa/slaw
26. Indigo Platform Documentation, https://indigo.readthedocs.io/_/downloads/en/docs/pdf/
27. sankalpsrv/Schematise: An LLM enabled XML generator for Indian laws in the LegalDocML and LegalRuleML formats - GitHub, https://github.com/sankalpsrv/Schematise
28. Table parsing · Issue #8 · Schematise-Lex-Data-Analysis/akoma-markup - GitHub, https://github.com/Schematise-Lex-Data-Analysis/akoma-markup/issues/8
29. Prajwalb0208/Akoma-Ntoso-Legal-Document-Extractor - GitHub, https://github.com/Prajwalb0208/Akoma-Ntoso-Legal-Document-Extractor
30. Bharat Scene Text Dataset (BSTD) - Emergent Mind, https://www.emergentmind.com/topics/bharat-scene-text-dataset-bstd
31. BhashaDaan - Bhashini, https://bhashini.gov.in/en
32. IndicNLP Resources - AI4Bharat, https://indicnlp.ai4bharat.org/pages/indicnlp-resources/
33. Tarento Joins Ekstep To Build The Pillar For National Language Translation Mission Via ULCA Platform, https://www.tarento.com/case-studies/tarento-joins-ekstep-to-build-the-pillar-for-national-language-translation-mission-via-ulca-platform/
34. sarvamai/sarvam-translate - Hugging Face, https://huggingface.co/sarvamai/sarvam-translate
35. :bookmark: The Indic NLP Catalog | indicnlp_catalog, https://ai4bharat.github.io/indicnlp_catalog/
36. Indigo Platform | OpenUp, https://openupsa.github.io/indigo.html